Submitted To : Mam Shumail Nasreen

Submitted By: FA23-BSE-036,FA23-BSE-056

Title : Project Report

Online Blood Bank Managment System(Project Report)

Course: Web Engineering Lab (B.Sc. Software Engineering)

Project Title: Online Blood Bank Management System

Live URL: https://github.com/laibariasat56/online-blood-bank

Submitted By: 1. Laiba Riasat (FA23-BSE-036)

2. Saadia Sajid (FA23-BSE-056)

Submitted To: Ms. Shumaila Nasreen

1. Project Introduction

The "Online Blood Bank Management System" is a full-stack web application designed to automate the operations of a blood bank. It facilitates the management of donor records, patient information, and blood requests, ensuring that life-saving information is accessible in real-time.

2. Problem Statement

Traditional manual record-keeping in blood banks is prone to errors and slow response times. Finding a specific blood group during an emergency is difficult without a centralized digital database. This system provides a streamlined platform to track donors and manage blood requests efficiently.

3. System Features

- **Admin Dashboard:** A summary view showing the total number of donors, patients, and requests.
- **Donor Management:** Capability to register new donors and maintain a list with their blood group and contact details.
- **Patient Management:** Maintaining records of patients registered within the system.
- **Blood Requests:** A tracking system for pending and completed blood requests.
- **Inventory Overview:** Monitoring the availability of various blood groups (A+, B+, O-, etc.).

4. Technology Stack

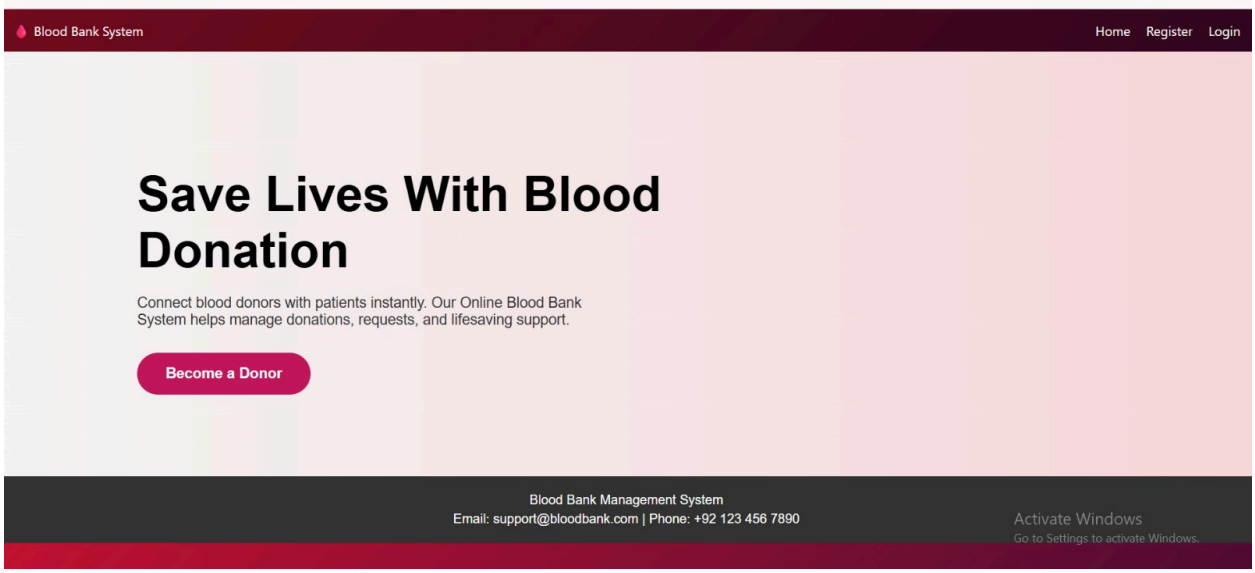
- **Frontend:** HTML5, CSS3 (Custom styling), and JavaScript.
- **Backend:** Node.js with Express.js framework.

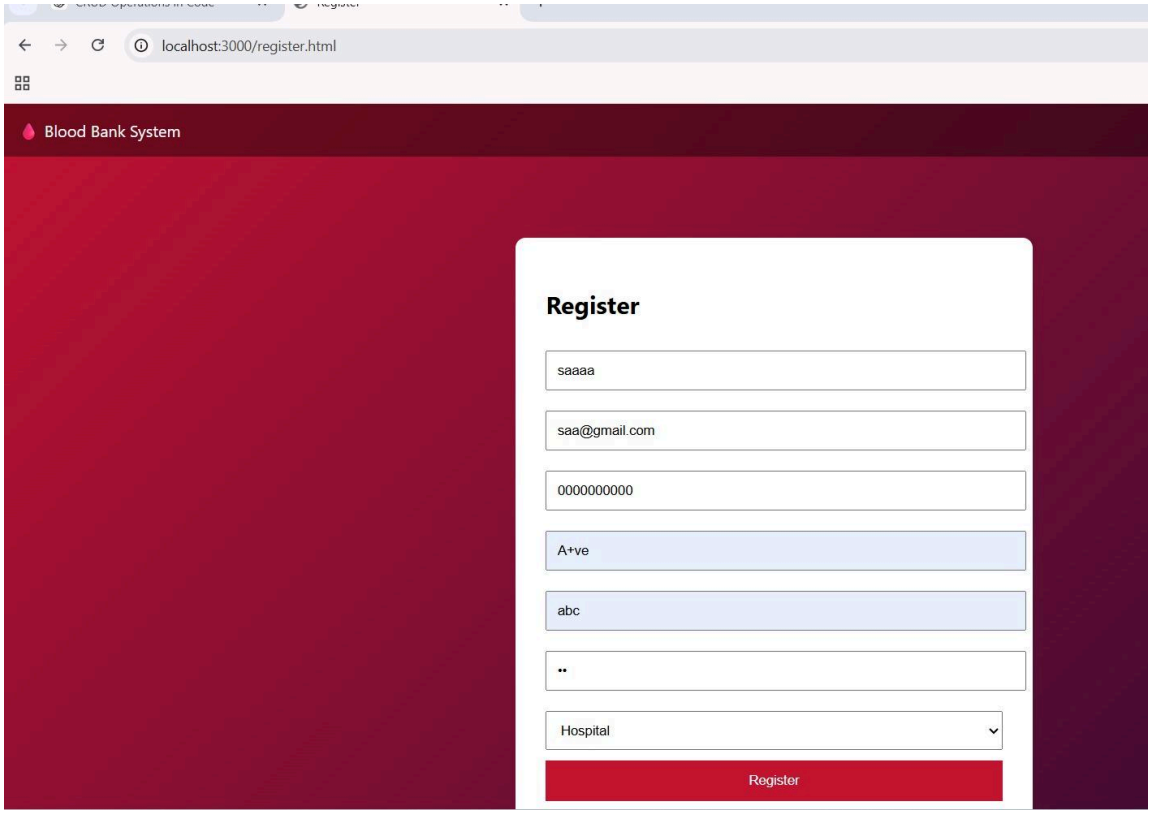
- **Database:** MySQL (Relational Database).
- **Deployment:** Railway Cloud.
- **Version Control:** Git & GitHub.

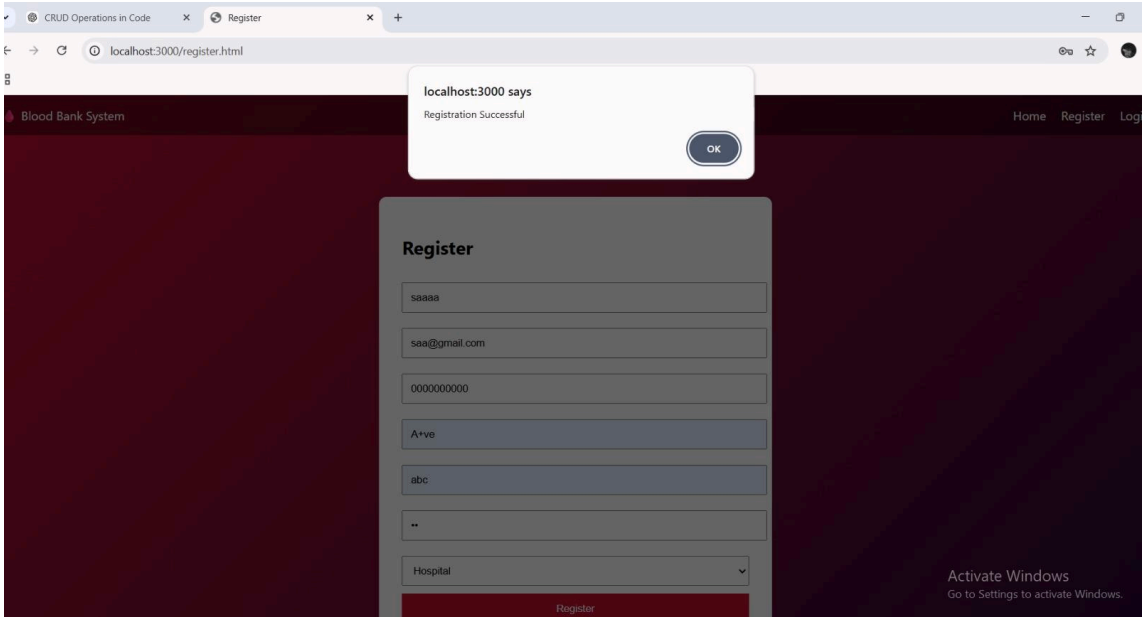
5. Frontend Design Explanation

The user interface is designed for simplicity and usability. It uses a sidebar for navigation and card-based layouts for the dashboard statistics.

- **Responsiveness:** The layout adjusts to different screen sizes.
- **Visual Elements:** Icons and color-coded cards are used to represent different data metrics.




localhost:3000/register.html
Blood Bank System
Register
saaaa
saa@gmail.com
0000000000
A+ve
abc
••
Hospital
Register


CRUD Operations in Code
Register
localhost:3000/register.html
localhost:3000 says
Registration Successful
OK
Blood Bank System
Home
Register
Register
saaaa
saa@gmail.com
0000000000
A+ve
abc
••
Hospital
Register
Activate Windows
Go to Settings to activate Windows.

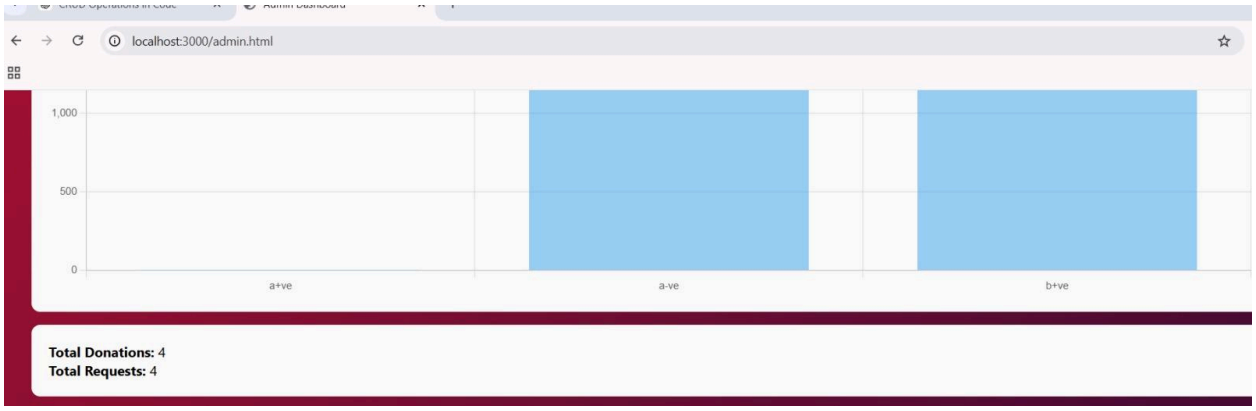
Donation Records

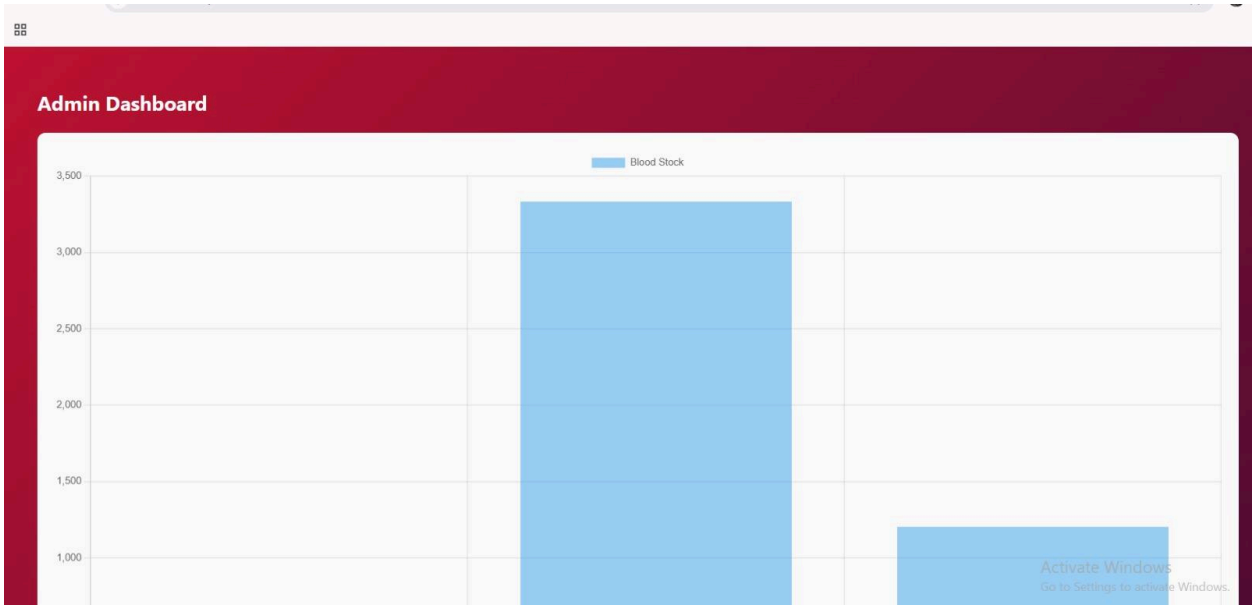
Name	Email	Blood Group	Quantity	Date
Hadi	hadi@gmail.com	a+ve	1	2026-03-01T19:00:00.000Z
ali	ali@gmail.com	a-ve	3333	2026-03-01T19:00:00.000Z
Mavia	mavia@gmail.com	b+ve	2	2026-03-01T19:00:00.000Z
Mavia	mavia@gmail.com	b+ve	1200	2026-03-01T19:00:00.000Z

Blood Requests

Name	Email	Blood Group	Quantity	Status	Action
Hanan	hanan@gmail.com	1	1	Pending	Approve
Laiba Riasat	laibaaaariasat56@gmail.com	a+	12000	Pending	Approve
Laiba Riasat	laibaaaariasat56@gmail.com	b+	1000	Pending	Approve
Laiba Riasat	laibaaaariasat56@gmail.com	a+	1200	Pending	Approve

Activate Windows
Go to Settings to activate Windows.





6. Backend Architecture & Coding Logic

The backend of the "Online Blood Bank Management System" is built using Node.js and the Express.js framework. It manages the communication between the user interface and the MySQL database through the following logic:

A. Database Configuration and Connectivity

The system uses the `mysql2` driver to establish a connection with the database.

- **Logic: A connection object is created using parameters such as `host`, `user`, `password`, and `database`.**
- **Implementation: The code uses `db.connect()` to verify the link. If the connection fails, it logs an error; otherwise, it confirms a successful connection to the MySQL server, allowing the application to perform data operations.**

B. Routing and Request Handling

The server uses Express Routes to handle different sections of the application (Donors, Patients, Requests).

- **Logic: For every page or action, a specific "Route" is defined. When a user clicks on a menu item like "Patients," the server receives a `GET` request.**
- **Implementation: The backend executes a SQL `SELECT` query to retrieve the current list of patients or donors. This data is then passed to the frontend to be displayed in the tables seen in the screenshots.**

C. Data Submission Logic (POST Requests)

When adding a new donor or patient, the system follows this logic:

1. **Extraction: The server extracts data from the input fields (Name, Blood Group, Contact) using `req.body`.**
2. **Validation: It ensures the data is not empty before processing.**
3. **Execution: An `INSERT INTO` SQL command is triggered to add the new record to the respective table.**
4. **Redirection: Once the database confirms the save, the logic redirects the user back to the updated list.**

D. Dashboard Analytics Logic

The main dashboard logic involves gathering summary data to display in the statistical cards.

- **Logic: The server runs multiple aggregate queries (like `COUNT(*)`) simultaneously. It counts the total entries in the `donors` table, `patients` table, and `blood_requests` table to show the live status of the system.**

db.js > ...

```javascript
const mysql = require("mysql2");

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "1234",
  database: "bloodbank"
});

db.connect(err => {
  if(err) console.log(err);
  else console.log("Connected to MySQL Database");
});

module.exports = db;
```

Route

adminRoutes.js M | userRoutes.js | admin.html M | style.css M | server.js M | README.md M

routes > adminRoutes.js > ...

```javascript
const express = require("express");
const router = express.Router();
const db = require("../db");

/* ================= INVENTORY ================= */
router.get("/inventory", (req, res) => {
  db.query("SELECT * FROM inventory", (err, result) => {
    res.json(result);
  });
});

/* ================= APPROVE REQUEST ================= */
router.post("/approve", (req, res) => {
  const { request_id, blood_group, quantity } = req.body;

  // Check available stock first
  db.query(
    "SELECT quantity FROM inventory WHERE blood_group=?",
    [blood_group],
    (err, result) => {
      if (result.length === 0) {
        return res.send("Blood group not available in inventory");
      }

      if (result[0].quantity < quantity) {
        return res.send("Not enough stock available");
      }

      // Update request status
      db.query(
        "UPDATE requests SET status='Approved' WHERE request_id=?",
        [request_id]
      );

      // Deduct inventory
      db.query(
        "UPDATE inventory SET quantity = quantity - ? WHERE blood_group=?",
```

Activate
Go to Setti

routes > JS adminRoutes.js > ...

```
router.get("/analytics", (req, res) => {
  });
});

/* ================= FULL DATA (DONATIONS + REQUESTS WITH JOIN)
================= */
router.get("/full-data", (req, res) => {
  db.query(
    `SELECT d.donation_id, u.name, u.email, d.quantity, d.blood_group,
d.donation_date
     FROM donations d
     JOIN users u ON d.donor_id = u.user_id`,
    (err, donations) => {
      db.query(
        `SELECT r.request_id, u.name, u.email, r.blood_group,
r.quantity, r.status
         FROM requests r
         JOIN users u ON r.hospital_id = u.user_id`,
        (err2, requests) => {
          res.json({ donations, requests });
        }
      );
```

7. Database Design (As per Proposal)

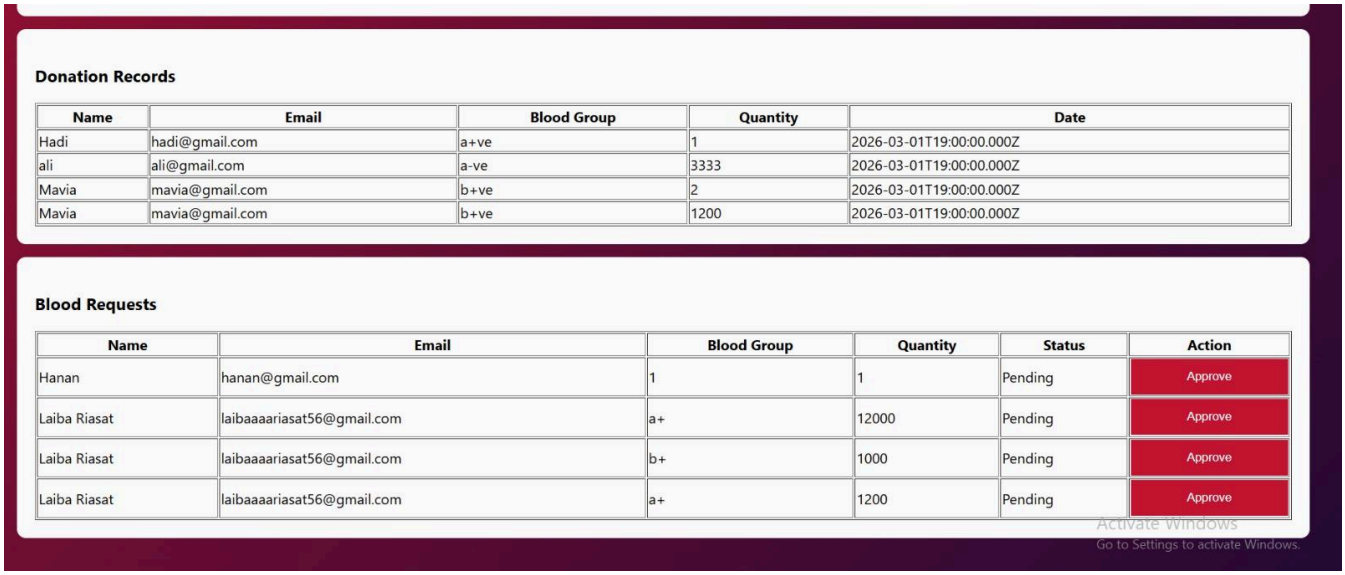
The database structure is strictly based on the initial project proposal to ensure data consistency:

- **Donors Table:** Stores donor_id, name, blood_group, and contact.
- **Patients Table:** Stores patient_id, name, and email.
- **Requests Table:** Stores request_id, blood_group, and status.

```
+--------------------+
| Tables_in_bloodbank |
+--------------------+
| camps              |
| donations          |
| inventory          |
| notifications      |
| requests           |
| users              |
+--------------------+
6 rows in set (0.00 sec)
```

8. CRUD Operations Implementation

- **Create:** Functionality to add new donors and patients through web forms.
- **Read:** Fetching data from the MySQL database to display in tabular format.
- **Update:** Logic to modify existing records of donors or patients.



Donation Records

Name	Email	Blood Group	Quantity	Date
Hadi	hadi@gmail.com	a+ve	1	2026-03-01T19:00:00.000Z
ali	ali@gmail.com	a-ve	3333	2026-03-01T19:00:00.000Z
Mavia	mavia@gmail.com	b+ve	2	2026-03-01T19:00:00.000Z
Mavia	mavia@gmail.com	b+ve	1200	2026-03-01T19:00:00.000Z

Blood Requests

Name	Email	Blood Group	Quantity	Status	Action
Hanan	hanan@gmail.com	1	1	Pending	Approve
Laiba Riasat	laibaaaariasat56@gmail.com	a+	12000	Pending	Approve
Laiba Riasat	laibaaaariasat56@gmail.com	b+	1000	Pending	Approve
Laiba Riasat	laibaaaariasat56@gmail.com	a+	1200	Pending	Approve

Activate Windows
Go to Settings to activate Windows.

9. GitHub & Deployment

The project is hosted on GitHub for version control and deployed via Railway for public access.

- **GitHub Repository:** https://github.com/laibariasat56/online-blood-bank
- **Deployment:** Automated deployment from the GitHub main branch.

laibariasat56 / online-blood-bank

Type /

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

online-blood-bank Public

Pin | Watch 0

main | 1 Branch | 0 Tags | Go to file t | Add file | Code

laibariasat56 Update README.md ✓ — f5f5e4c · 1 minute ago — 6 Commits

public	Update admin.html	3 hours ago
routes	Update adminRoutes.js	3 hours ago
.gitignore	Initial commit - Online Blood Bank System	19 hours ago
README.md	Update README.md	2 minutes ago
db.js	Initial commit - Online Blood Bank System	19 hours ago
package-lock.json	Initial commit - Online Blood Bank System	19 hours ago
package.json	Initial commit - Online Blood Bank System	19 hours ago
server.js	Update server.js	3 hours ago

laibariasat56 / online-blood-bank

Type / to search

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

Files

main

Go to file t

- public
  - css
  - js
  - admin.html
  - donor.html
  - hospital.html
  - index.html
  - inventory.html
  - login.html
  - register.html
- routes
  - adminRoutes.js
  - userRoutes.js

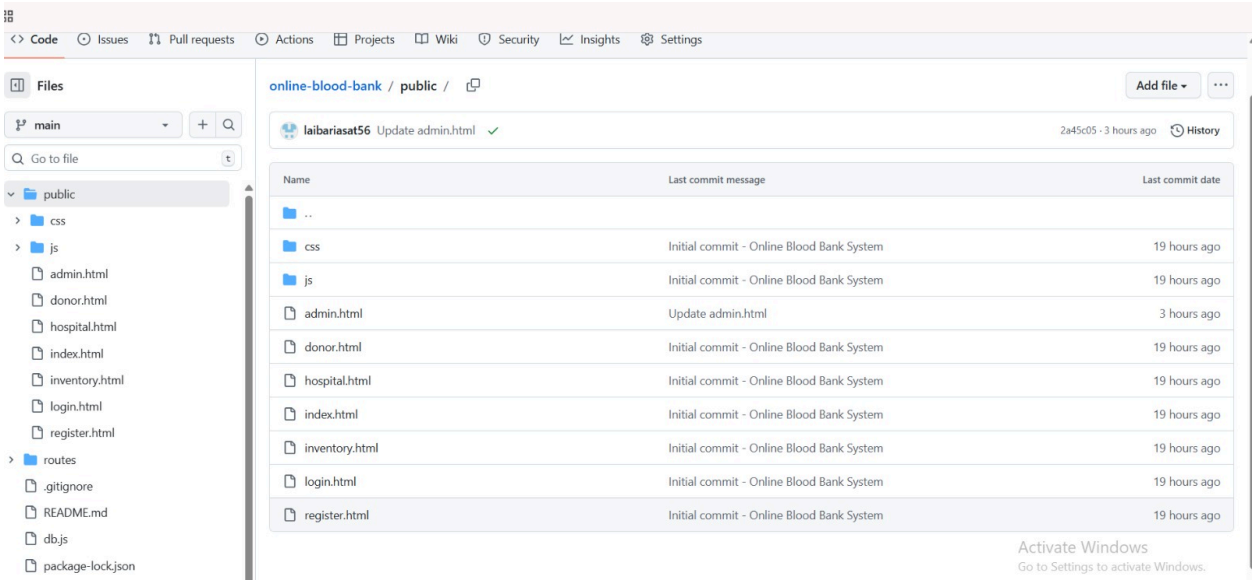
online-blood-bank / routes /

Add file

laibariasat56 Update adminRoutes.js ✓ — 45a836d · 3 hours ago — History

Name	Last commit message	Last commit date
..		
adminRoutes.js	Update adminRoutes.js	3 hours ago
userRoutes.js	Initial commit - Online Blood Bank System	19 hours ago

Activate Windows
Go to Settings to activate Windows.



10. Conclusion & Future Enhancements

The project successfully meets the requirements of a full-stack web application.

Future Scope:

- Adding an automated email/SMS notification system for blood requests.
- Implementing an advanced search filter for location-based donor matching.

1.